

KL UNIVERSITY

Department of Computer Science and Engineering

CAPSTONE PROJECT REPORT

CLOUD - 158

Load Balancer Health Checking That Reflects Real Service Capability

Submitted By

S.No	Roll No.	Name
1	2300030394	MADDINENI PRAVALLIKA
2	2300030746	YARABARLA SARAN KUMAR
3	2300030908	MOHAMMAD MUSHTHAQUDDIN
4	2300033416	ANE SAI SPOORTHY

Under the Guidance of the Department of CSE-I Project Committee

Dr. K. PRAKASH

Project Guide

Academic Year 2026 – 2027

Table of Contents

1. Abstract	3
2. Introduction	4
3. Existing Algorithms	5
4. Limitations of Existing Approaches	6
5. Proposed System	7
6. Workflow	8
7. Tools and Technologies Used	10
8. Conclusion.....	10

1. Abstract

Most web applications run on a group of backend servers instead of a single server. A load balancer is placed in front of this group to decide which server should handle each incoming request. For this decision to be correct, the load balancer must know which servers are able to serve traffic at a given time. At present, this is done through a basic health check, in which the load balancer sends a ping or opens a TCP connection to check if the server responds.

This type of check only shows whether the server process is running. It does not show whether the server can complete the request. A server can respond to a ping even when its database connection has failed, its cache is not reachable, or one of its dependent services is not working. In such cases, the load balancer continues to send requests to a server that is unable to process them, and the request fails.

This project presents a health checking method that examines more than the basic ping used by TCP or HTTP checks. The proposed method checks whether the server is actually able to serve the request, and not just whether it is reachable. A test setup was created with multiple backend servers. Normal traffic was generated, and failures were introduced in server dependencies such as the database. The results show that the proposed method identifies unhealthy servers earlier and reduces the number of failed requests reaching the user.

2. Introduction

Web applications such as shopping sites, banking applications, and food delivery platforms must remain available even when many users access them at the same time. To manage this load, applications are not run on a single server. Instead, several copies of the same application are run on different servers, and a load balancer is placed in front of them. The task of the load balancer is to send each incoming request to a server that can handle it correctly.

To make this decision, the load balancer uses health checks. A health check is a small test that the load balancer performs on each backend server at regular intervals to decide whether that server should continue receiving traffic. If a server fails the health check a few times, it is removed from the pool until it responds correctly again.

During the study of this topic, it was observed that most commonly used health checking methods work at a shallow level. They check whether the server is reachable, but do not check whether the server is functioning correctly from end to end. This creates a gap between what the load balancer assumes about a server and what the server can actually deliver. Reducing this gap is the main objective of this project.

2.1 Objectives

- To study commonly used load balancing and health checking algorithms.
- To identify situations where existing methods fail to detect an unhealthy server.
- To design a health check that verifies dependency and service-level readiness, and not only network reachability.

2.2 Scope of the Project

This project focuses on improving load balancer health checks by verifying the actual service capability of backend servers. It includes implementing a capability-aware health checking mechanism using HAProxy, Docker, and multiple backend servers, and comparing it with traditional TCP and HTTP health checks under different failure conditions.

3. Existing Algorithms

Load balancing plays an important role in improving the performance and availability of distributed applications. To distribute incoming client requests efficiently and ensure that only healthy backend servers receive traffic, load balancers use different traffic distribution algorithms and health checking methods. Before proposing the capability-aware health checking approach, it is important to understand the commonly used algorithms and techniques. This section describes the existing algorithms, their working principles, advantages, and limitations.

3.1 Traffic Distribution Algorithms

1. **Round Robin:**

Round Robin distributes incoming requests equally among all backend servers in a circular order. It is simple to implement and works well when all servers have similar capacity. However, it does not consider the current workload of each server.

2. **Weighted Round Robin:**

Weighted Round Robin assigns a weight to each server based on its processing capacity. Servers with higher weights receive more requests than servers with lower weights. This improves resource utilization but does not adapt to real-time server conditions.

3. **Least Connections:**

The Least Connections algorithm forwards each new request to the server with the fewest active connections. It is suitable for applications with long-running sessions and helps distribute the workload more efficiently. However, it does not consider server health or dependency failures.

3.2 Health Checking Methods

1. **TCP Health Check:**

The TCP Health Check verifies whether a TCP connection can be established with the backend server. If the connection is successful, the server is marked as healthy. It is fast and simple but cannot detect application-level or dependency failures.

2. **HTTP Health Check:**

The HTTP Health Check sends an HTTP request to a predefined endpoint, such as `/health`, and checks whether the server returns a successful response (HTTP 200). It provides better validation than TCP checks but may still fail to detect internal issues like database or service failures.

These methods are fast and simple, which is the main reason they are widely used. However, both methods only confirm that the server's network connection and web server process are active. Neither method checks whether the components the application depends on, such as a database, a cache layer, or another internal service, are working correctly.

4. Limitations of Existing Approaches

The main weakness identified in existing health checks is that they treat server reachability and server capability as the same thing, although in practice they are not. The following situations were observed during the study, where this gap leads to real problems:

- Database connection exhausted: The web server is running and responds to the TCP or HTTP ping, but its connection pool to the database is full. Any request that needs data will fail, even though the health check passes.
- Slow or unresponsive dependency: The server depends on another internal service, such as a payment or authentication service, that has become slow. The health check does not test that dependency, so the server still appears healthy.
- Cache layer down: If a service depends on a cache such as Redis and the cache becomes unavailable, requests may still succeed but take much longer, which lowers the quality of service without failing the basic health check.
- Partial deployment issues: During a rolling deployment, a server may respond to pings before it has finished loading its configuration, which causes the first few requests to fail.

In each of these cases, the load balancer continues to send traffic to a server that is technically running but is not able to serve the request correctly. From the perspective of the user, this appears as failed transactions, timeouts, or inconsistent behaviour, even though the monitoring dashboard may still show all servers as healthy.

5. Proposed System

The proposed system introduces a **capability-aware health checking mechanism** that evaluates whether a backend server is capable of processing client requests before the load balancer routes traffic to it. Unlike traditional health checks, which only verify that the server is running, the proposed method also checks the readiness of the application and the availability of its critical resources.

The health check is performed in three stages:

5.1 Readiness Check

The Readiness Check verifies that the application has started successfully and is ready to receive client requests. It ensures that the application has completed its startup process and is prepared to handle incoming traffic.

5.2 Dependency Check

The Dependency Check verifies the availability of essential resources such as the MySQL database and monitors the availability of database connections. It also checks whether the application has sufficient resources, such as available threads, to process incoming requests.

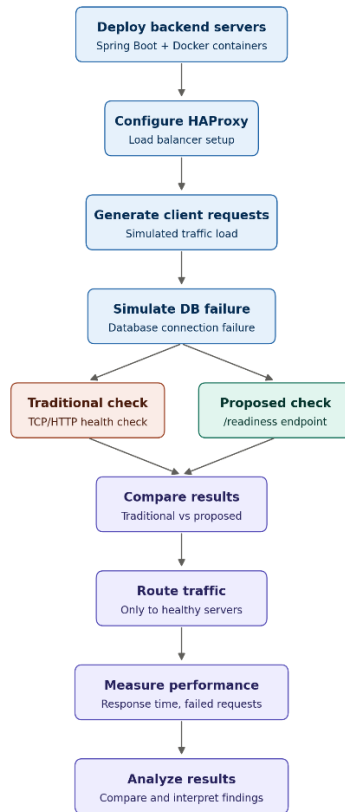
5.3 Capability Check

The Capability Check performs a lightweight application-level operation, such as executing a simple database query or verifying database connectivity, to confirm that the backend server can successfully process client requests. This avoids routing traffic to servers that are running but are unable to provide the requested service.

A backend server is considered healthy only when it successfully passes all three stages. If any stage fails, the server is temporarily removed from the HAProxy load balancer until it becomes healthy again. This approach prevents requests from being routed to unhealthy servers and improves the reliability of the overall system.

6. Workflow

To evaluate the proposed capability-aware health checking system, the workflow shown in figure was followed. The system was implemented using HAProxy, Spring Boot, Docker, and MySQL. The existing health check and the proposed readiness-based health check were tested under the same conditions to compare their performance and reliability.



6.1 Step-by-step Explanation

1. **Deploy Backend Servers** – Multiple Spring Boot backend server instances were deployed using Docker containers and connected to a common MySQL database.
2. **Configure HAProxy** – HAProxy was configured as the load balancer to distribute incoming client requests among the backend servers. Initially, the traditional **/health** endpoint was used for health monitoring.
3. **Implement the Readiness Check** – A **/readiness** endpoint was implemented in each backend server to verify application readiness, database connectivity, and the server's ability to process client requests.
4. **Simulate Failure Conditions** – Failure scenarios, such as database connection pool exhaustion, were introduced while keeping the application process running to simulate servers that were reachable but unable to process requests.

5. **Perform Health Checks** – HAProxy periodically sent health check requests to the backend servers. The responses from the traditional **/health** endpoint and the proposed **/readiness** endpoint were observed and compared.
6. **Route Client Requests** – Backend servers that successfully passed the readiness check remained in the active server pool and received client requests. Servers that failed the readiness check were temporarily removed from the load balancing pool.
7. **Evaluate System Performance** – The performance of both approaches was compared by observing metrics such as failed requests, response time, and the ability to detect unhealthy servers under failure conditions.

7. Tools and Technologies Used

1. **HAProxy** – Used as the load balancer to distribute incoming client requests among multiple backend servers and to implement both the traditional **/health** check and the proposed **/readiness** health check.
2. **Spring Boot** – Used to develop the backend applications and implement the **/health** and **/readiness** endpoints for health monitoring.
3. **Docker & Docker Compose** – Used to containerize the backend applications, HAProxy, and MySQL, making it easy to deploy and manage the complete testing environment.
4. **MySQL** – Used as the backend database to simulate database connectivity and connection pool failure scenarios.
5. **Git & GitHub** – Used for version control and collaboration among team members throughout the project development.

8. Conclusion

This project focused on improving the reliability of load balancing by introducing a capability-aware health checking mechanism. Unlike traditional TCP and HTTP health checks, which only verify whether a backend server is reachable, the proposed approach also checks the application's readiness, database connectivity, and ability to process client requests. This helps the load balancer make more accurate routing decisions and prevents traffic from being sent to servers that are running but unable to provide the required service.

The proposed system was implemented using Spring Boot, HAProxy, Docker, and MySQL and evaluated in a controlled laboratory environment. The implementation demonstrates how capability-aware health checks can improve server selection and enhance the overall reliability and availability of load-balanced applications.